# MiniProject 2: Training and Evaluation Practices

Bogdan Rudnitski Olivier Ren Maxim Qu

**Abstract**

This project investigates how optimization and regularization affect logistic regression for spam classification using the UCI Spambase dataset. We implemented logistic regression with mini-batch gradient descent and analysed the effects of batch size, learning rate, training duration, weight initialization, and L2 regularization through ten-fold cross-validation. We also used scikit-learn to construct an L1 regularization path, tracking coefficient values and the number of nonzero coefficients across regularization strengths. The reported final L2 models achieved approximately 89.3% test accuracy. In a separate ten-fold comparison, L1 logistic regression achieved its highest mean accuracy of 93.48% at $C \approx 2.59$. This project reveals the impact of regularization on validation error and its difference with the training error.

## 1 Introduction

In this project, we use the UCI Spambase dataset, which contains 4,601 emails with 57 numerical features to perform spam classificaiton. We model the probability that an email is spam using logistic regression and implement its training procedure from scratch using mini-batch stochastic gradient descent. We first examine how batch size, learning rate, training duration, and weight initialization influence learning, using ten-fold cross-validation on the training set to compare configurations. We then investigate L2 regularization by varying its strength and comparing training and validation performance to explore the bias–variance trade-off. Finally, we use scikit-learn to study L1 regularization through coefficient paths, feature sparsity, and cross-validation accuracy. Together, these experiments determine how optimization and regularization influence predictive performance, model complexity, and interpretability.

## 2 Methods

### 2.1 Data Preprocessing

First, we shuffled the dataset by generating random indices and applying the same permutation for both the features ($\mathbf{X}$) and labels ($\mathbf{y}$) to preserve the association between them. Then, we selected the first 5% of the dataset for training, and the remaining 95% for testing. Finally, we normalized both datasets using the mean and standard deviation taken exclusively from the training set (prevent data leakage) and add the bias term.

### 2.2 Logistic Regression Implementation

We implemented a binary logistic regression model with mini-batch stochastic gradient descent (SGD) from scratch using separate Python classes for the model and the SGD optimizer.

We trained our model by minimizing the cross-entropy loss:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N}\left[y_i \ln(1+e^{-z_i}) + (1-y_i)\ln(1+e^{z_i})\right], \quad z_i = \mathbf{x}_i^\top \mathbf{w} \tag{1}$$

The mini-batch SGD optimizer updates our model's weights:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} L_B$$

The optimizer and LR model take hyperparameters as inputs which can be tuned.

### 2.3 L2 Regularization Convention

We used the following convention $\mathcal{L}(\mathbf{w}) + \lambda|\mathbf{w}|_2^2$.

The regularization strength $\lambda$ controls the magnitude of the penalty, with $\lambda = 0$ recovering the unregularized model. We excluded the bias term from the regularization penalty, as it should not be penalized. Thus, the gradient of the penalty term, $2\lambda\mathbf{w}$, was incorporated into the mini-batch SGD update rule:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta\left(\nabla_{\mathbf{w}}\mathcal{L}_B + 2\lambda\mathbf{w}\right)$$

Since the loss function was only used for evaluation and visualization, we kept it unregularized, allowing us to compare the cross-entropy loss consistently across different regularization strengths.

### 2.4 Cross-Validation Protocol

For the cross-validation, we split the training dataset into K "folds", i.e. K equal sized datasets, where K-1 folds will be used for training and the remaining fold will be used for validation. Therefore, we normalized the data with the mean and standard deviation of the K-1 training folds, avoiding data leakage.

We defined a range of values for several hyperparamater (learning rate, batch size, number of epochs, regularization strength and initial weight definition). For each combination of these hyperparamater values, we ran K training, obtaining cross-entropy and accuracy for each, and then compute the mean and standard deviation on both. This protocol is used for both task 2 and 3, we used K=10 for both.

Tasks 2 and 3, both use suggested hyperparameter ranges. We also have weight initialization: "Zero" which sets all weights to 0, "Ones" which sets them to 1, "Uniform" which sets the weights in a uniform distribution between -1 and 1, and "Normal" which sets the weights in a normal distribution with mean 0, standard deviation of 1.

## 3 Results

### 3.1 Logistic Regression with SGD

We trained our logistic regression model with and without L2 regularization ($\lambda = 0.01$ and $\lambda = 0$ respectively), keeping all other hyperparamaters set to default values. During training, we recorded both the unregularized cross-entropy loss and the accuracy after each epoch, as shown in Figure 1 and Figure 2, respectively.

The unregularized model achieved lower training loss and slightly higher accuracy, while the regularized model plateaued earlier and showed greater fluctuations in accuracy. This is consistent

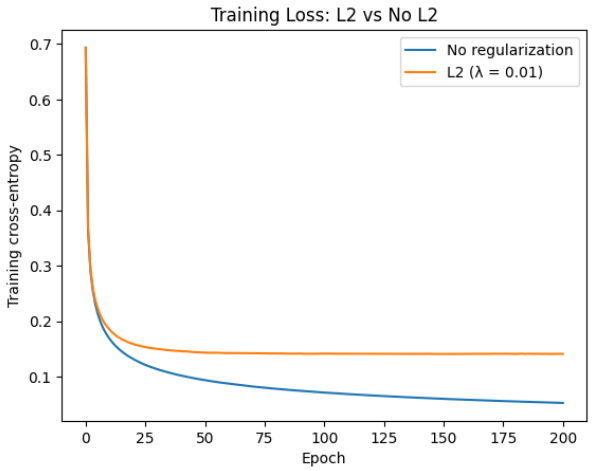



Figure 1: Training cross-entropy loss with and without L2 regularization ($\lambda = 0.01$ and $\lambda = 0$).

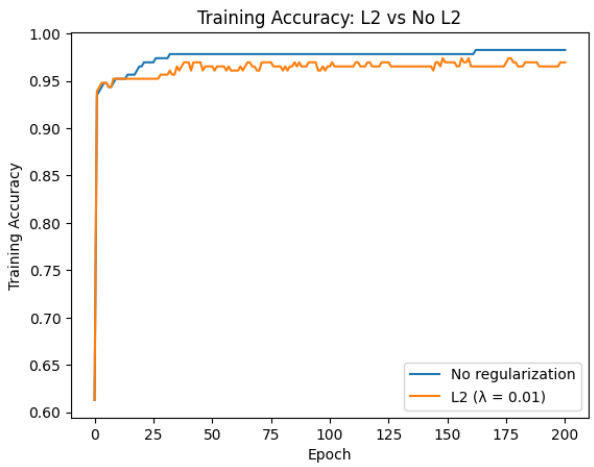



Figure 2: Training accuracy with and without L2 regularization ($\lambda = 0.01$ and $\lambda = 0$).

with the effects of L2 regularization, which aims to reduce overfitting by simplifying the model, which thus performs worse on the training set.

We also examined how batch size influences the training stability. We compared full SGD (batch size = 1) with mini-batch SGD by recording training cross-entropy loss across both wieght updates and epochs, as shown in Appendix C (Figure 10 and Figure 11). Figure 10 shows substantially fluctuating training loss curve for SGD, whereas mini-batch SGD with batch size 16 and 64 decreased more smoothly. These fluctuations are less apparent in Figure 11, since loss is recorded only at the end of each epoch. Smaller batch sizes achieve lower training loss in fewer epochs, as they perform more weight updates per epoch. However, when comparing by weight updates, batch size 16 and 64 generally achieved lower and much more stable loss than batch size 1.

### 3.2 Hyperparameter Tuning with K-Fold Cross-Validation

We tuned our hyperparameters using 10-fold cross-validation, computing the means and the standard deviations of the cross-entropy error and accuracy across the folds. This resulted in 190 hyperparameter configurations. We identified the 10 best configurations, ranked by lowest mean cross-entropy error and highest mean accuracy, shown in Appendix A (Table A1 and Table A2).

The best configurations converged to around the same values (0.9 for accuracy and 0.3 for cross-entropy error). To better understand the influence of individual hyperparamaters, we also identified the 10 worst configurations, ranked by highest mean cross-entropy and lowest mean accuracy (Table A3 and Table A4). The worse performing configurations tended to have very low learning rates, particularly 0.0001. This suggests that some configurations did not have time to converge within the specified number of epochs due to smaller update steps. Conversely, there are many learning rates of 1.0 among the best configurations.

The best configuration we identified was the 2nd lowest in mean cross-entropy error (0.3085) and had the highest mean accuracy (0.9304). It had batch size 16, learning rate 1.0, 200 epochs, and an uniform weight generation method.

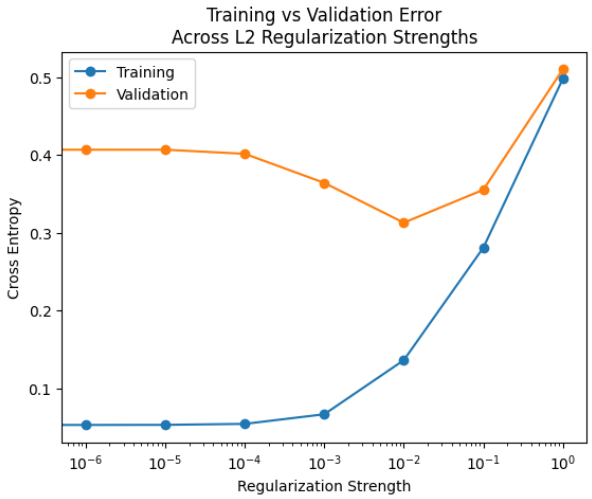



Figure 3: Training cross-entropy error vs validation cross-entropy error given different regularization strength

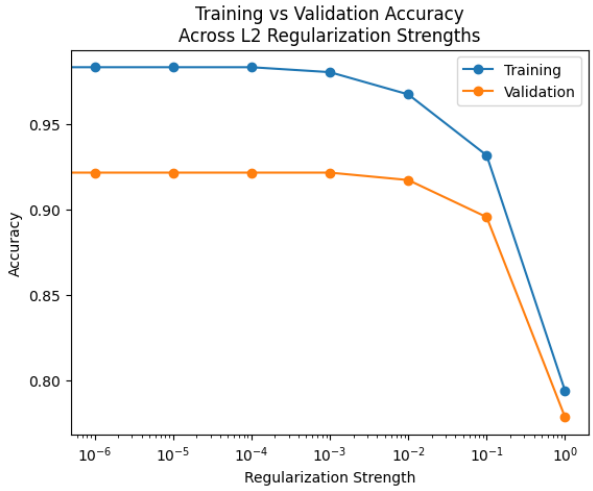



Figure 4: Training accuracy vs validation accuracy given different regularization strength

### 3.3 Bias–Variance Trade-Off via $\lambda$ Sweep

For each suggested $\lambda$ value, we computed training and validation cross-entropy, and training and validation accuracy with our cross-validation model. As seen in Figure 3, training error is smallest when $\lambda$ is smaller because the extra model expressivity can fit the training data better, but have worse validation error due to overfitting (model not generalizing well to unseen data). High $\lambda$ have high validation and training error due to underfitting. We can observe the U shape of cross validation in Figure 3, which shows the middle ground between overfitting and underfitting. Similarly, Figure 4 shows how training and validation accuracy change with regularization strength, with both decreasing with strong regularization. This is why we chose 0.1 as the best $\lambda$ value. Retraining on the entire training set and using the model on the test set results in a cross-entropy error of 0.3495 and accuracy of 0.8897.

We also did this process with the best hyperparameters values obtained from Task 2, with the corresponding cross-entropy and accuracy curves presented in Appendix B (Figure 8 and Figure 9). We chose $\lambda$ of 0.01 instead because it had lowest validation error and highest validation accuracy. Using this model on the test set, resulted in a cross-entropy error of 0.3308 and accuracy of 0.9004.

### 3.4 L1-Regularized Logistic Regression and the Regularization Path

A regularization path shows how the model's coefficients change as the regularization strength varies. For L1 regularization, strong penalties can force coefficients to zero, essentially excluding those features from the model. Scikit-learn controls regularization through $C$: smaller values impose stronger regularization, while larger values weaken the penalty. We fitted models over 30 logarithmically spaced values of $C$ from $10^{-4}$ to $10^{4}$.

Figure 9 shows the coefficient paths for the ten features with the largest absolute coefficients at the weakest regularization setting. At small C, all displayed coefficients are zero. As C increases, features begin contributing to the model, with positive coefficients increasing the predicted probability of spam and negative coefficients decreasing it, holding other features constant.

Among the displayed features, `char_freq_!`, `char_freq_$`, and `word_freq_remove` become nonzero

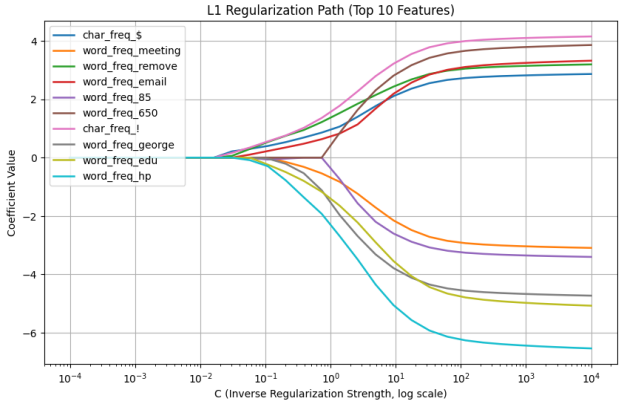



Figure 5: L1 regularization paths for the ten features with the largest absolute coefficients at the weakest regularization setting ($C = 10^4$).

relatively early with positive coefficients. This suggests that these features serve as effective indicators for spam detection, even under high regularization penalties. Features including `word_freq_hp`, `word_freq_george`, and `word_freq_edu` develop negative coefficients, indicating associations with legitimate emails.

Figure 10 shows how the number of nonzero coefficients changes across the path. Strong regularization eliminates all feature coefficients, leaving predictions dependent on the intercept. As regularization weakens, the number of selected features increases, eventually reaching approximately 57. The increase is not strictly monotonic: individual features can enter or leave the model as other coefficients change.

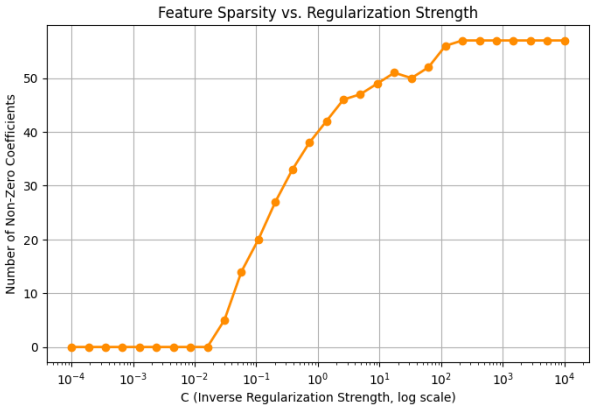



Figure 6: Number of nonzero feature coefficients across L1 regularization strengths, illustrating increasing sparsity as $C$ decreases.

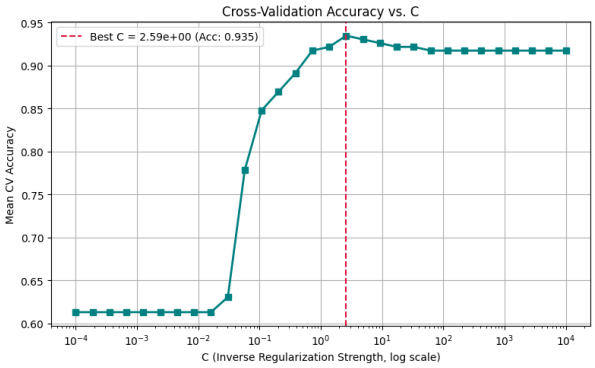



Figure 7: Mean 10-fold cross-validation accuracy across L1 regularization strengths, with the optimal $C \approx 2.59$ indicated by the dashed line.

Sparse models are easier to interpret because their predictions depend on fewer features. However, excessive sparsity can remove useful information and cause underfitting. Figure 11 illustrates this trade-off: mean ten-fold cross-validation accuracy is approximately 61% under the strongest regularization, peaks at 93.48% around $C \approx 2.59$, and then decreases toward 92% as regularization weakens further. These results favor an intermediate penalty over either extreme.

# 4 Discussion and Conclusion

The bias-variance tradeoff in task 3 could be clearly seen in the first run where we observed a U shaped in the validation error showing a good middle ground between high bias and high variance. This cannot be observed for the second run where we used the optimal hyperparameters from task 2, this might be due to low $\lambda$ granularity, we could have more values for L2 strengths if we did this again to observe this tradeoff better. A limitation during our training with L2 regularization is that we did not have enough hyperparameter granularity. What we could have done is that after finding a good hyperparameter combination, try to other values that are near that combination (a iterative grid search). A potential next step could be to use non basis functions to add non linear features. This could potentially reveal non-linear relationship between features and the target (spam/non-spam). L2 encourages smaller coefficients, whereas L1 can eliminate coefficients entirely, producing a model that depends on fewer features.

# Statement of Contributions

Bogdan implemented the logistic regression model and mini-batch SDG code for Task 1, helped develop the cross-validation code in Tasks 2 and 3, and wrote the report with help from teammates. Olivier did the data preprocessing for Task 1 and wrote code and report for Tasks 2 and 3 with help from teammates. Maxim implemented the L1 regularization for Task 4, and helped write the report.

### Statement on the use of LLMs and other resources

No LLMs were used by the team members.

# A Cross-Validation Results

Table A1: Top 10 configurations ranked by lowest validation cross-entropy ($\lambda = 0.01$).

| Batch | LR | Epochs | Init. Weights | CE Mean | CE SD | Acc Mean | Acc SD |
|---|---|---|---|---|---|---|---|
| 64 | 1.0000 | 50 | Normal | 0.307 | 0.241 | 0.909 | 0.066 |
| 16 | 1.0000 | 200 | Uniform | 0.309 | 0.258 | 0.930 | 0.052 |
| 64 | 1.0000 | 100 | Zero | 0.309 | 0.245 | 0.913 | 0.055 |
| 64 | 1.0000 | 50 | Uniform | 0.310 | 0.266 | 0.917 | 0.049 |
| 16 | 0.1000 | 100 | Normal | 0.312 | 0.250 | 0.913 | 0.061 |
| 64 | 1.0000 | 100 | Normal | 0.312 | 0.259 | 0.917 | 0.057 |
| 16 | 1.0000 | 200 | Normal | 0.312 | 0.254 | 0.922 | 0.054 |
| 16 | 1.0000 | 100 | Ones | 0.312 | 0.252 | 0.922 | 0.058 |
| 1 | 0.0100 | 200 | Normal | 0.312 | 0.252 | 0.917 | 0.057 |
| 1 | 0.0100 | 100 | Normal | 0.313 | 0.251 | 0.917 | 0.057 |

Table A2: Top 10 configurations ranked by highest validation accuracy ($\lambda = 0.01$).

| Batch | LR | Epochs | Init. Weights | CE Mean | CE SD | Acc Mean | Acc SD |
|---|---|---|---|---|---|---|---|
| 16 | 1.0000 | 200 | Uniform | 0.309 | 0.258 | 0.930 | 0.052 |
| 1 | 0.1000 | 200 | Zero | 0.319 | 0.252 | 0.926 | 0.048 |
| 16 | 1.0000 | 200 | Ones | 0.314 | 0.257 | 0.926 | 0.052 |
| 64 | 1.0000 | 50 | Zero | 0.318 | 0.251 | 0.922 | 0.058 |
| 1 | 0.1000 | 200 | Ones | 0.330 | 0.290 | 0.922 | 0.047 |
| 16 | 1.0000 | 100 | Ones | 0.312 | 0.252 | 0.922 | 0.058 |
| 16 | 1.0000 | 200 | Normal | 0.312 | 0.254 | 0.922 | 0.054 |
| 64 | 1.0000 | 100 | Ones | 0.314 | 0.257 | 0.922 | 0.051 |
| 1 | 0.1000 | 50 | Normal | 0.336 | 0.252 | 0.917 | 0.060 |
| 1 | 0.0100 | 50 | Zero | 0.316 | 0.256 | 0.917 | 0.057 |

Table A3: Bottom 10 configurations ranked by highest validation cross-entropy ($\lambda = 0.01$).

| **Batch** | **LR** | **Epochs** | **Init. Weights** | **CE Mean** | **CE SD** | **Acc Mean** | **Acc SD** |
|---|---|---|---|---|---|---|---|
| 64 | 0.0001 | 50 | Ones | 4.556 | 2.000 | 0.613 | 0.071 |
| 64 | 0.0001 | 100 | Ones | 4.538 | 1.994 | 0.613 | 0.071 |
| 16 | 0.0001 | 50 | Ones | 4.516 | 1.987 | 0.613 | 0.071 |
| 64 | 0.0001 | 200 | Ones | 4.502 | 1.983 | 0.613 | 0.071 |
| 16 | 0.0001 | 100 | Ones | 4.458 | 1.969 | 0.617 | 0.064 |
| 64 | 0.0010 | 50 | Ones | 4.397 | 1.948 | 0.617 | 0.064 |
| 16 | 0.0001 | 200 | Ones | 4.345 | 1.931 | 0.617 | 0.064 |
| 64 | 0.0001 | 50 | Normal | 4.225 | 5.592 | 0.535 | 0.123 |
| 64 | 0.0010 | 100 | Ones | 4.224 | 1.888 | 0.622 | 0.070 |
| 16 | 0.0010 | 100 | Normal | 4.030 | 2.915 | 0.565 | 0.075 |

Table A4: Bottom 10 configurations ranked by lowest validation accuracy ($\lambda = 0.01$).

| **Batch** | **LR** | **Epochs** | **Init. Weights** | **CE Mean** | **CE SD** | **Acc Mean** | **Acc SD** |
|---|---|---|---|---|---|---|---|
| 64 | 0.0001 | 50 | Uniform | 2.567 | 1.089 | 0.474 | 0.141 |
| 16 | 0.0001 | 100 | Uniform | 1.834 | 0.742 | 0.474 | 0.124 |
| 64 | 0.0010 | 200 | Normal | 3.358 | 1.631 | 0.478 | 0.101 |
| 64 | 0.0001 | 100 | Normal | 3.294 | 1.270 | 0.491 | 0.148 |
| 64 | 0.0010 | 200 | Uniform | 2.016 | 1.164 | 0.504 | 0.128 |
| 16 | 0.0001 | 200 | Normal | 2.892 | 1.306 | 0.509 | 0.120 |
| 16 | 0.0001 | 50 | Uniform | 1.909 | 0.868 | 0.513 | 0.153 |
| 64 | 0.0010 | 50 | Normal | 3.073 | 1.007 | 0.517 | 0.163 |
| 64 | 0.0001 | 100 | Uniform | 2.145 | 0.993 | 0.517 | 0.092 |
| 16 | 0.0001 | 50 | Normal | 2.869 | 1.119 | 0.522 | 0.179 |

## B L2 Regularization with Selected Hyperparameters

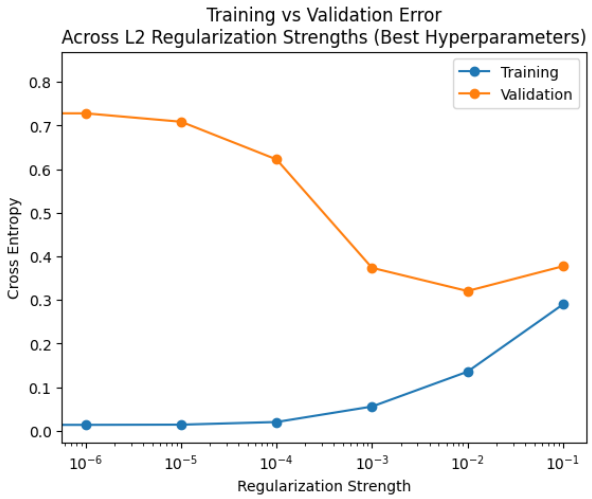



Figure 8: Training cross-entropy error vs validation cross-entropy error given different regularization strength with best hyperparameters

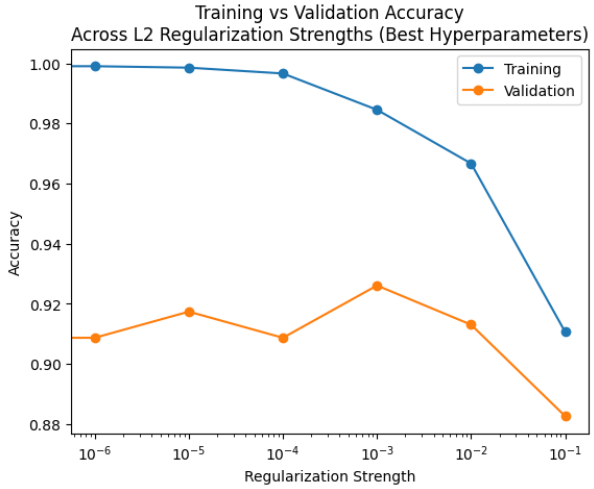



Figure 9: Training accuracy vs validation accuracy given different regularization strength with best hyperparameters

## C Training Loss Across Batch Sizes




Figure 10: Training cross-entropy loss by weight updates with different batch sizes.

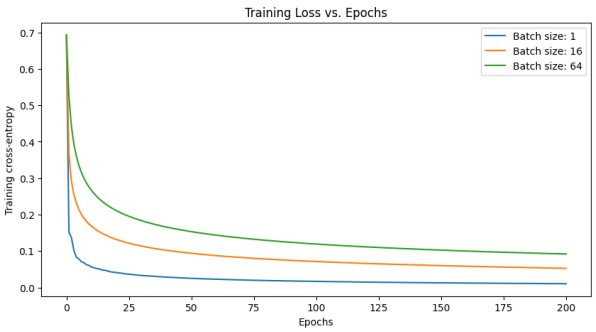



Figure 11: Training cross-entropy loss by epochs with different batch sizes.